Node.js
Practical No:- 01

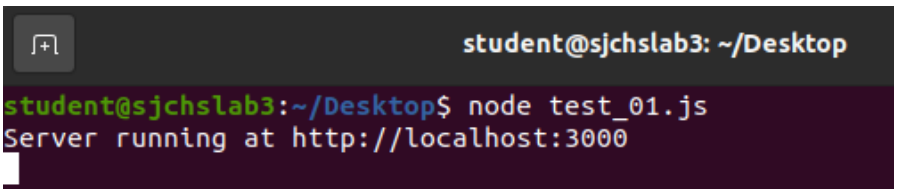
Aim:- To create a basic HTTP server using Node.js that handles multiple routes such as Home, About, Contact, and Services.

Code:-

```
const http = require('http');

const server = http.createServer((req, res) => {
    console.log(req.url); // debug

    // Ignore favicon request
    if (req.url === '/favicon.ico') {
        res.writeHead(204);
        res.end();
        return;
    }

    // HOME PAGE
    if (req.url === '/') {
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        res.end(
            'HOME PAGE\n\n' +
            'Welcome to our Node.js Web Server!\n' +
            'This server demonstrates basic routing.\n' +
            'Use the following routes:\n' +
            ' /about -> About Page\n' +
            ' /contact -> Contact Information\n' +
            ' /services -> Services We Offer\n'
        );
    }

// ABOUT PAGE
else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(
        'ABOUT PAGE\n\n' +
        'This application is built using Node.js.\n' +
        'Node.js allows us to run JavaScript on the server.\n' +
        'In this project, we demonstrate:\n' +
        '- Creating a server\n' +
        '- Handling routes\n' +
        '- Sending responses to clients\n'
    );
}
```

```
// CONTACT PAGE
else if (req.url === '/contact') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(
        'CONTACT PAGE\n\n' +
        'Developer Name: John Doe\n' +
        'Email: support@example.com\n' +
        'Phone: +91 9876543210\n' +
        'Office Hours: 10 AM - 6 PM IST\n'
    );
}

// SERVICES PAGE
else if (req.url === '/services') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(
        'SERVICE PAGE\n\n' +
        'We provide the following services:\n' +
        '- Web Development\n' +
        '- Node.js Backend Development\n' +
        '- API Integration\n' +
        '- Web Hosting Setup\n' +
        '- Technical Support and Maintenance\n'
    );
}
else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end(
        '404 - PAGE NOT FOUND\n\n' +
        'The page you are looking for does not exist.\n' +
        'Try:\n' +
        ' / -> Home\n' +
        ' /about -> About Us\n' +
        ' /contact -> Contact\n' +
        ' /services -> List of Services\n'
    );
}
});

server.listen(3000, () => {
    console.log('Server running at http://localhost:3000');
});
```

Output:-



localhost:3000

```
HOME PAGE

Welcome to our Node.js Web Server!
This server demonstrates basic routing.
Use the following routes:
 /about -> About Page
 /contact -> Contact Information
 /services -> Services We Offer
```

localhost:3000/about

```
ABOUT PAGE

This application is built using Node.js.
Node.js allows us to run JavaScript on the server.
In this project, we demonstrate:
- Creating a server
- Handling routes
- Sending responses to clients
```

localhost:3000/contact

```
CONTACT PAGE

Developer Name: John Doe
Email: support@example.com
Phone: +91 9876543210
Office Hours: 10 AM - 6 PM IST
```

localhost:3000/services

```
SERVICE PAGE

We provide the following services:
- Web Development
- Node.js Backend Development
- API Integration
- Web Hosting Setup
- Technical Support and Maintenance
```

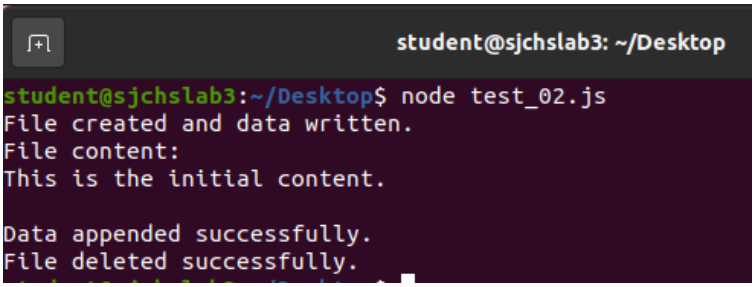
PRACTICAL NO:- 02

AIM:- To understand and use the Node.js File System (fs) module for performing file operations such as reading, writing, appending, and deleting files.

Code:-

```
const fs = require('fs');

// 1. Write to a file
fs.writeFile('example.txt', 'This is the initial content.\n', (err) => {
   if (err) throw err;
   console.log('File created and data written.');

  // 2. Read the file
   fs.readFile('example.txt', 'utf8', (err, data) => {
      if (err) throw err;
      console.log('File content:');
      console.log(data);

     // 3. Append data to the file
     fs.appendFile('example.txt', 'This text is appended.\n', (err) => {
         if (err) throw err;
         console.log('Data appended successfully.');

         // 4. Delete the file
         fs.unlink('example.txt', (err) => {
            if (err) throw err;
            console.log('File deleted successfully.');
         });
      });
   });
});
```

Output:-



Express.js
PRACTICAL NO. 03

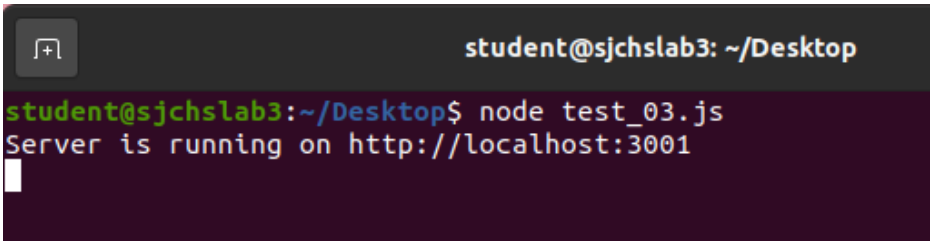
AIM:- TO DEMONSTRATE THE USE OF BUILT-IN, CUSTOM, AND THIRD-PARTY MIDDLEWARE IN EXPRESS.JS.

CODE:-

```
const express = require("express");
const morgan = require("morgan");
const app = express();

app.use(express.json());
app.use(morgan("dev"));

app.use((req, res, next) => {
   console.log("Custom Middleware Executed");
   next();
});

app.get("/", (req, res) => {
   res.send("Middleware Example");
});

app.listen(3001, () => {
   console.log("Server is running on http://localhost:3001");
});
```

Install morgan in Terminal:
npm install express
npm install morgan

Output:-



Middleware Example

Practical 4:
AIM:- RESTful Routing & API Endpoints (GET, POST, PUT, DELETE) using postman.

1. Prerequisites

Before you begin, ensure you have Node.js and npm (Node Package Manager) installed. You can verify this by running:

- node -v
- npm -v

2. Installing Express via Command Line

Create a project folder:

>>> mkdir postman_project
>>> cd postman_project

1. Initialize the project: This creates a package.json file.

   >>> npm init -y

2. Install Express:

   >>> npm install express

3. Creating the Local Script

Create a file named *index.js* in your project folder and paste the following code. This script sets up a basic server with one GET route and one POST route.

JavaScript code

```
// index.js
const express = require("express");
const app = express();
const PORT = 3000;
```

```
// Middleware to parse JSON
app.use(express.json());

// Temporary in-memory data (students)
let students = [
  { id: 1, name: "Alen", age: 22 },
  { id: 2, name: "BluBerry", age: 21 }
];

// 👉 GET all students
app.get("/students", (req, res) => {
  res.status(200).json(students);
});

// 👉 GET student by ID
app.get("/students/:id", (req, res) => {
  const student = students.find(s => s.id == req.params.id);
  if (!student) {
    return res.status(404).json({ message: "Student not found"
});
  }
  res.json(student);
});
// 👉 POST (Add new student)
app.post("/students", (req, res) => {
  const newStudent = {
    id: students.length + 1,
    name: req.body.name,
    age: req.body.age
  };
  students.push(newStudent);
  res.status(201).json(newStudent);
});

app.patch("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const student = students.find(u => u.id === id);
```

```
  if (!student) {
    return res.status(404).json({ message: "User not found"
});
  }

  // Update only provided fields
  Object.assign(student, req.body);

  res.json({
    message: "User updated partially",
    student
  });
});

// 👉 PUT (Update student)
app.put("/students/:id", (req, res) => {
  const student = students.find(s => s.id == req.params.id);
  if (!student) {
    return res.status(404).json({ message: "Student not found"
});
  }

  student.name = req.body.name;
  student.age = req.body.age;

  res.json(student);
});




// 👉 DELETE student
app.delete("/students/:id", (req, res) => {
  students = students.filter(s => s.id != req.params.id);
  res.json({ message: "Student deleted successfully" });
});
```

```
// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Running the Script

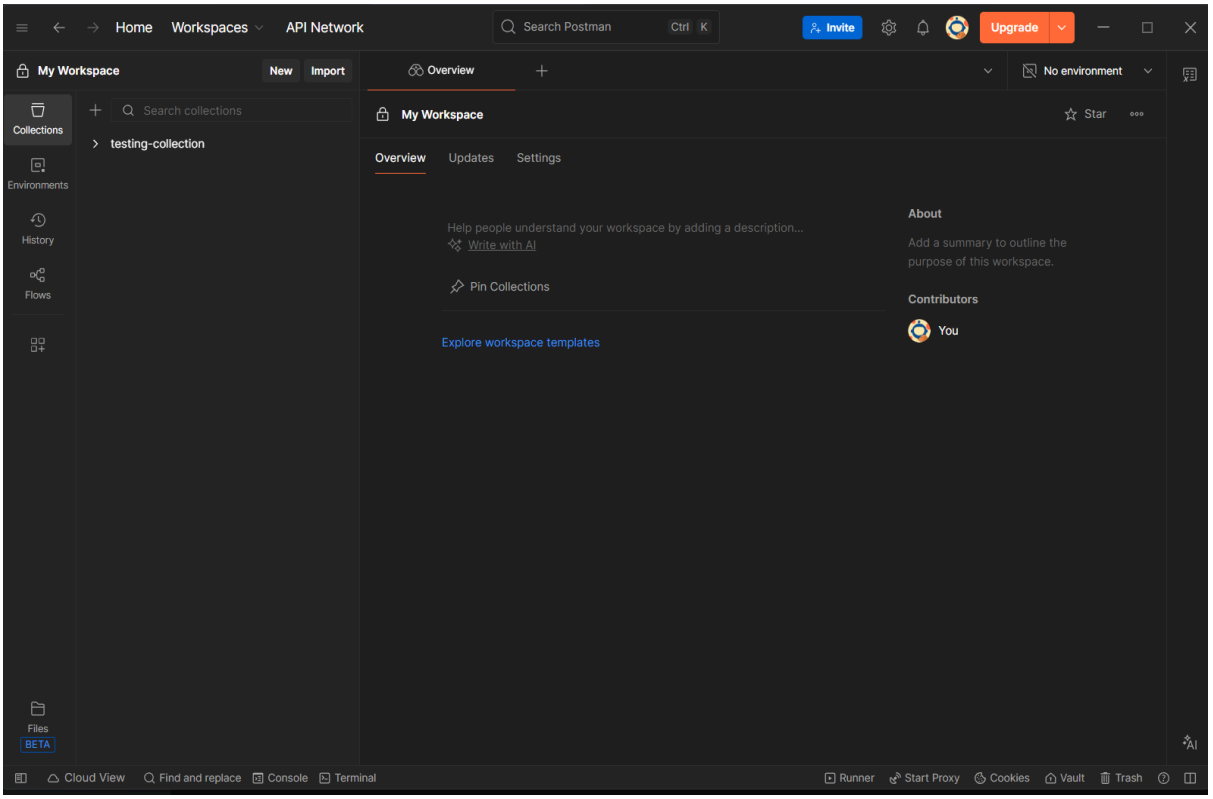
To start your server, run the following command in your terminal:

>>> node index.js

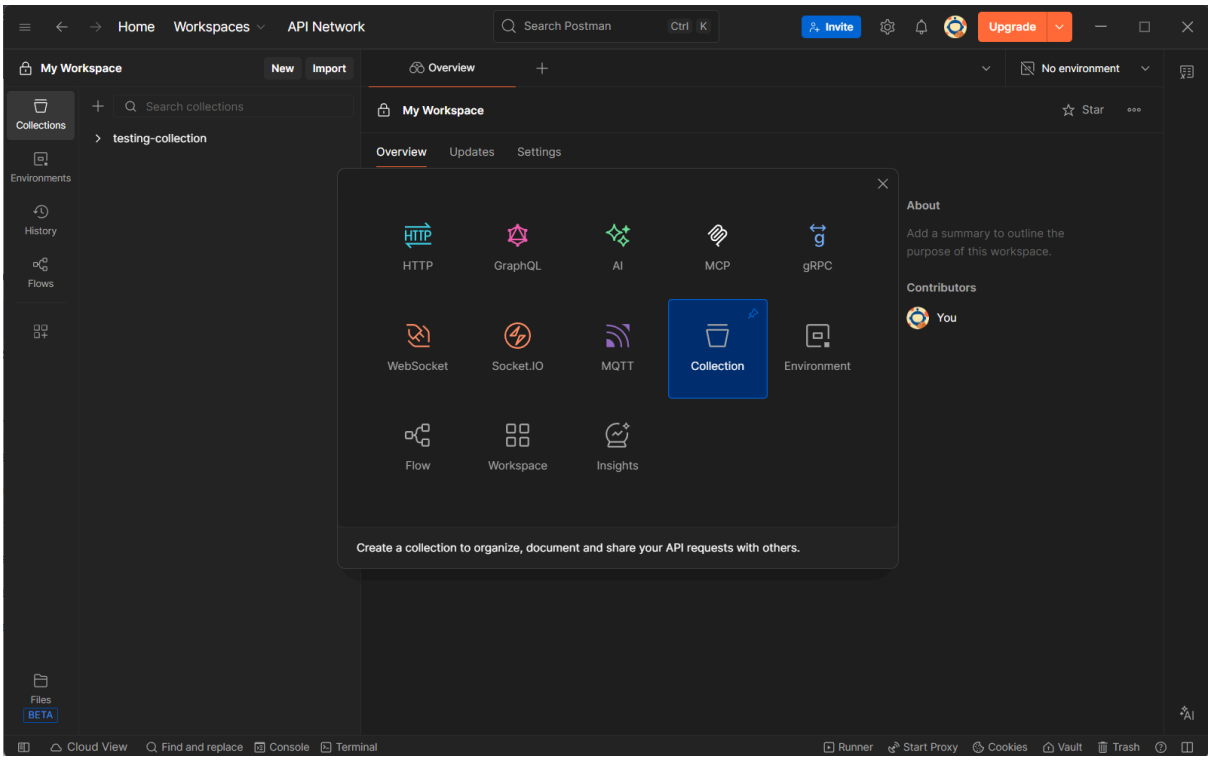
You should see: Server is running at http://localhost:3000.

5. Testing *GET* and *POST* Requests

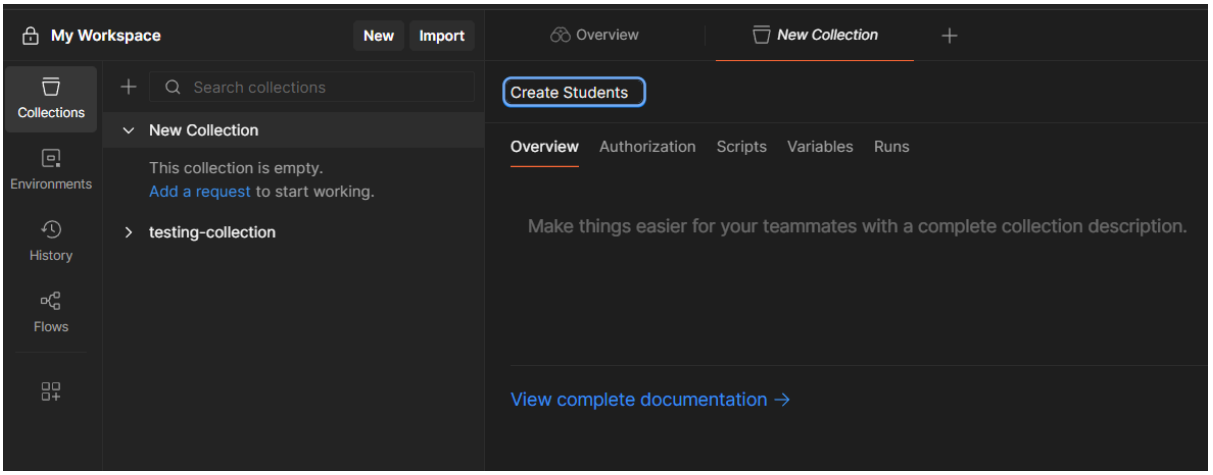
1. In Postman, click the Collections tab



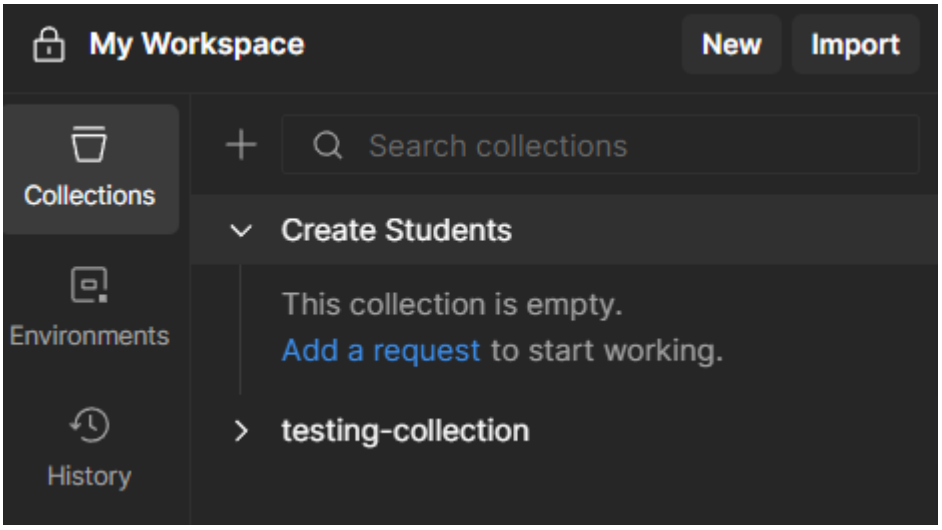
2. Then click on New button -> then click on Collection icon



3. Name the collection of your own
Example : Create Students

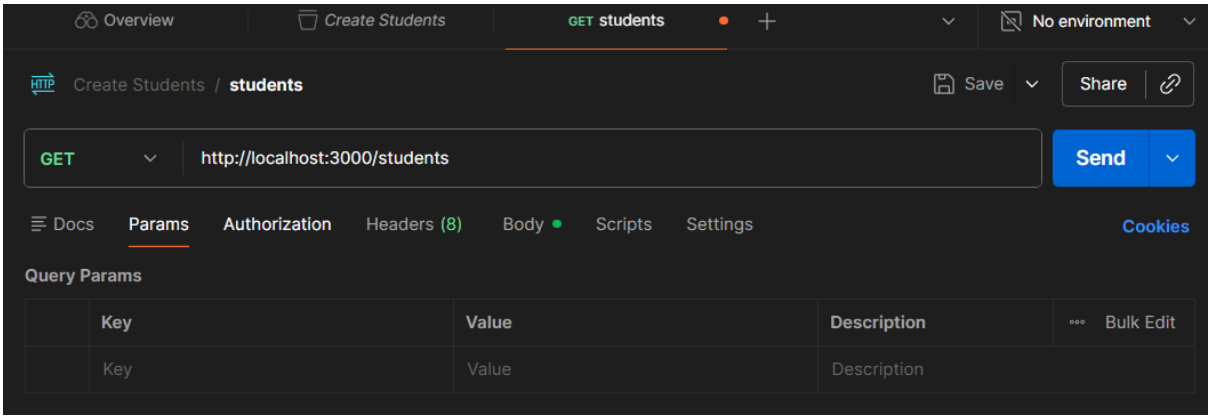


4. Click on "Add a request"

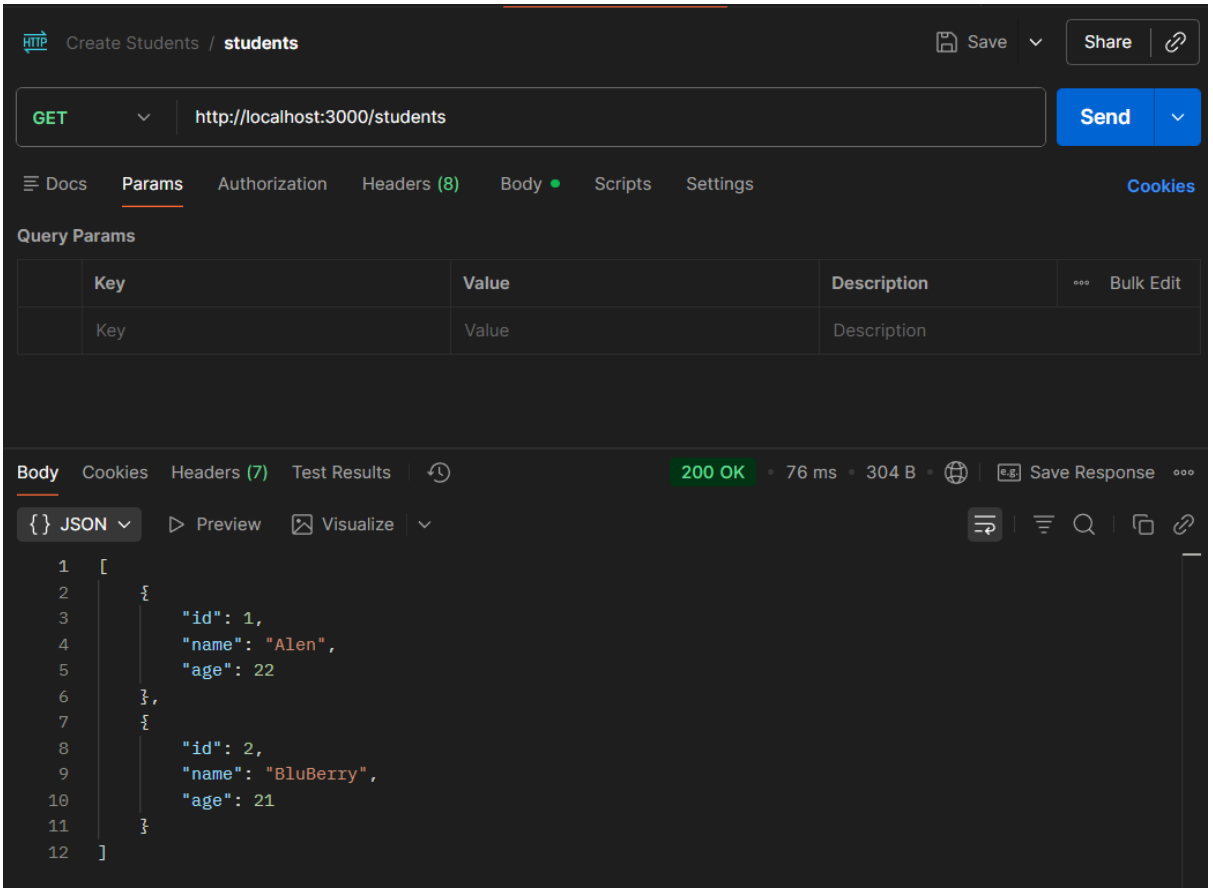


Testing the GET Request

5. Enter the local host link to the input field
   http://localhost:3000/students



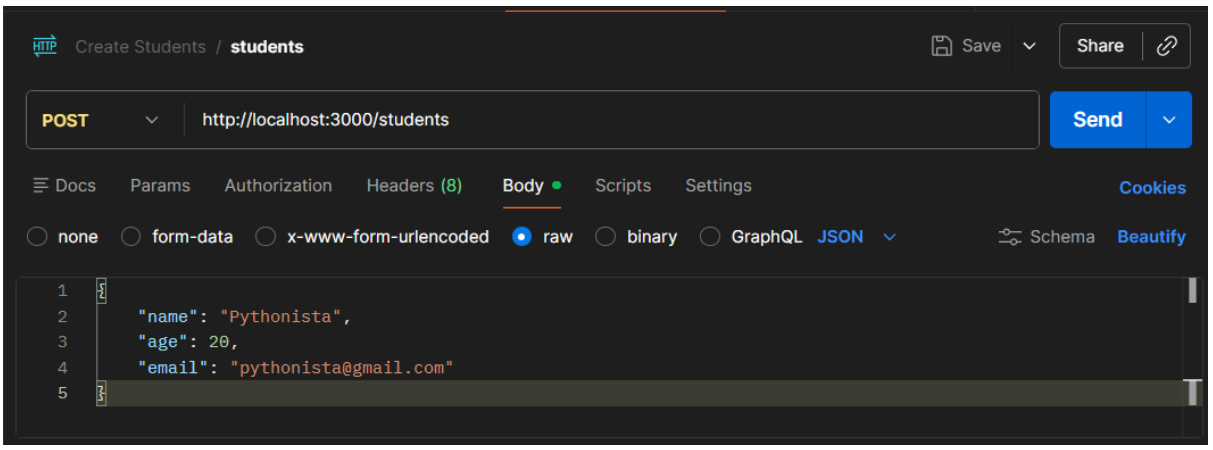
6. Click on Send button it throws this output as shown in the image



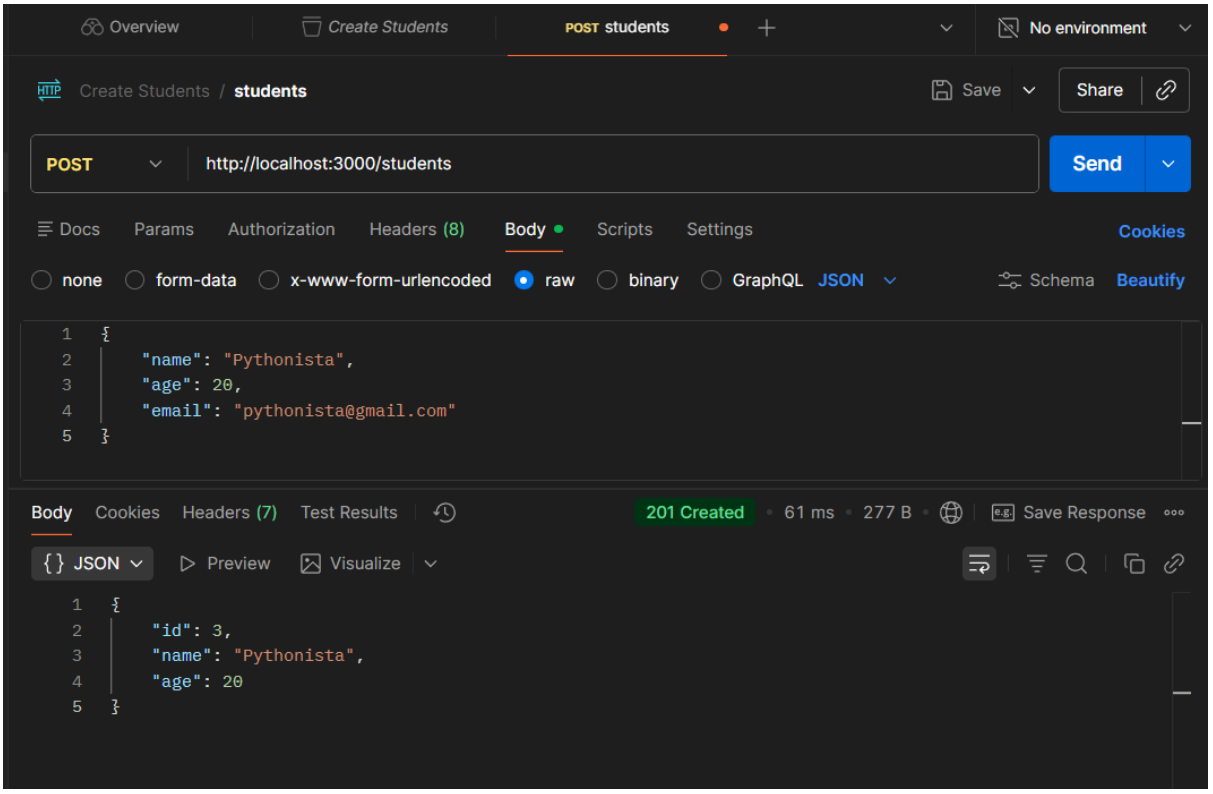
Testing the POST Request

1. Click on POST and click on Body tab paste the data

```
{
    "name": "Pythonista",
    "age": 20,
    "email": "pythonista@gmail.com"
}
```

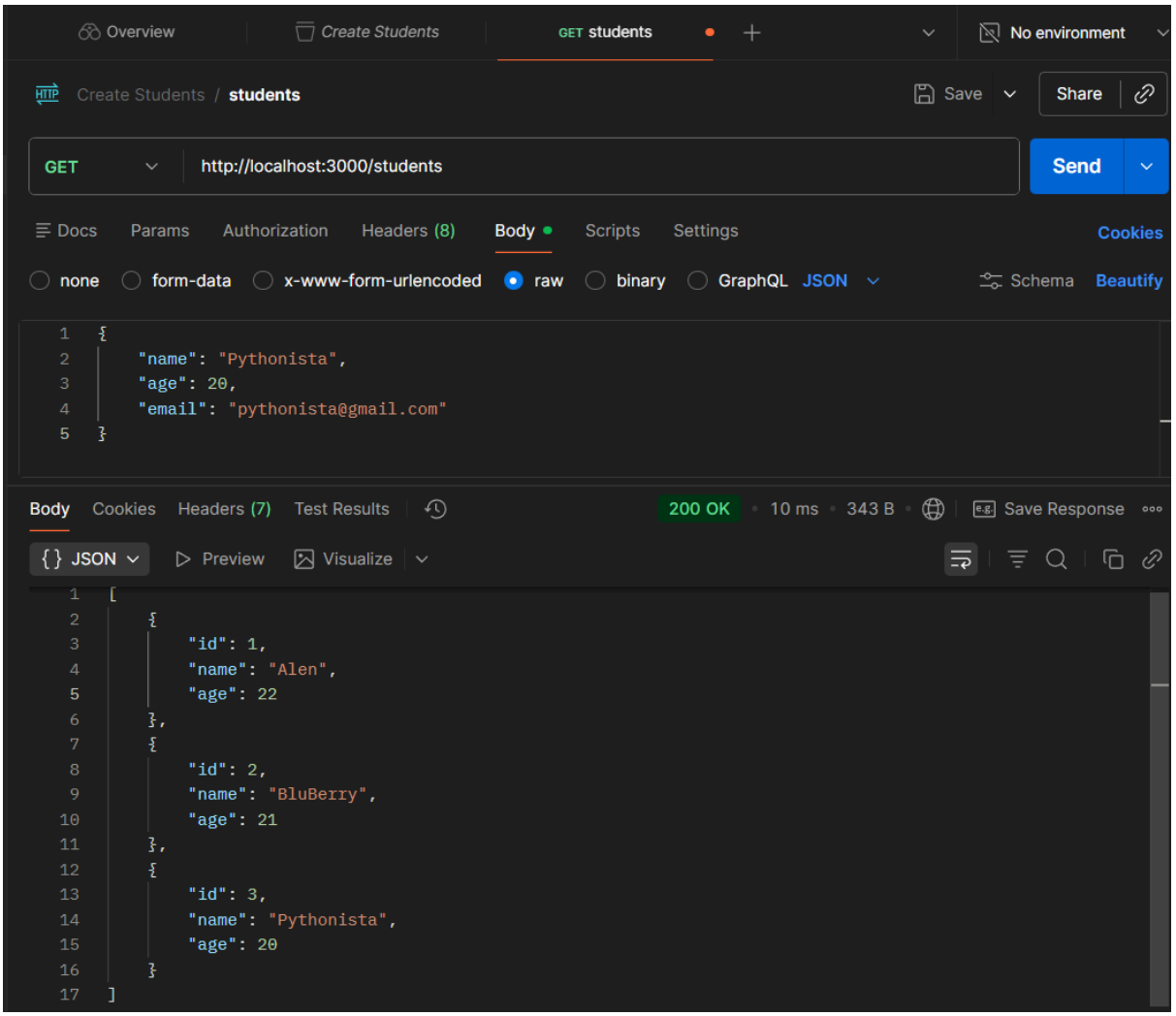


2. Click send button then the output must be same as the image shown with 201 created status code

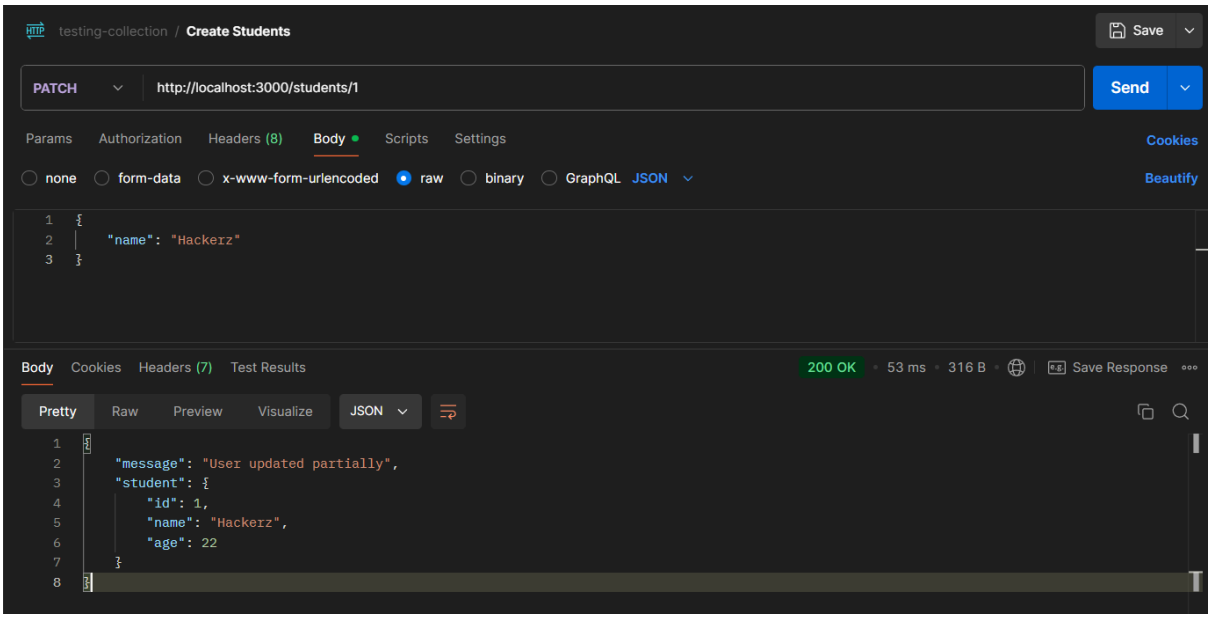


3. Click on GET method again to verify the data inserted or not

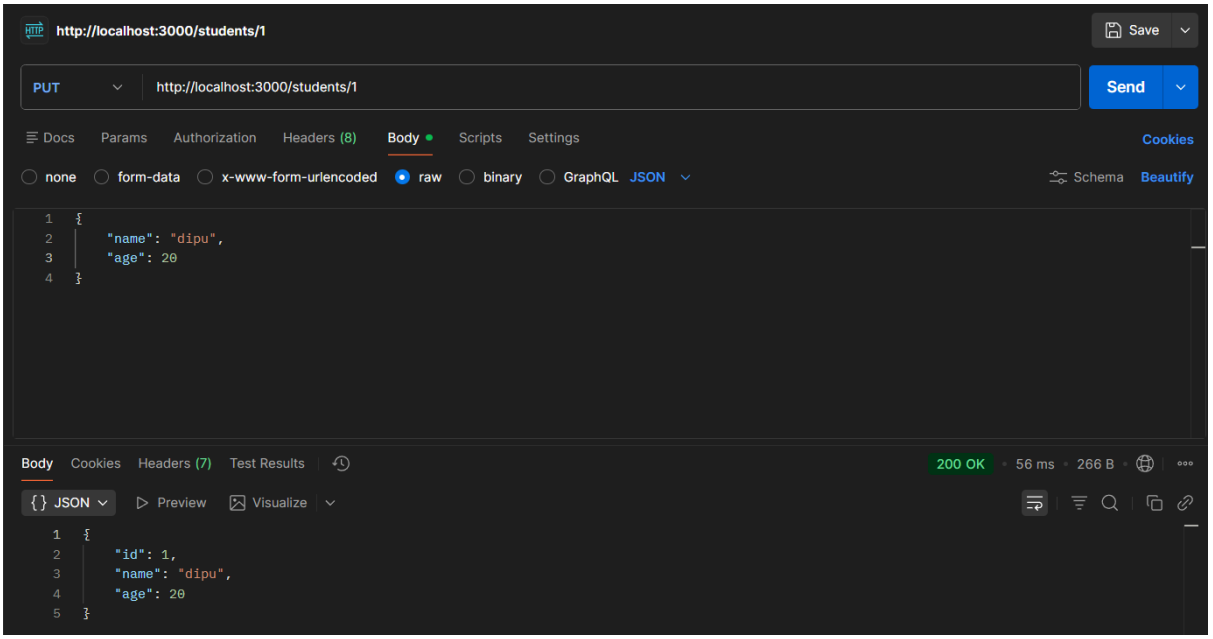
If the json data returned with the added data then the API is working



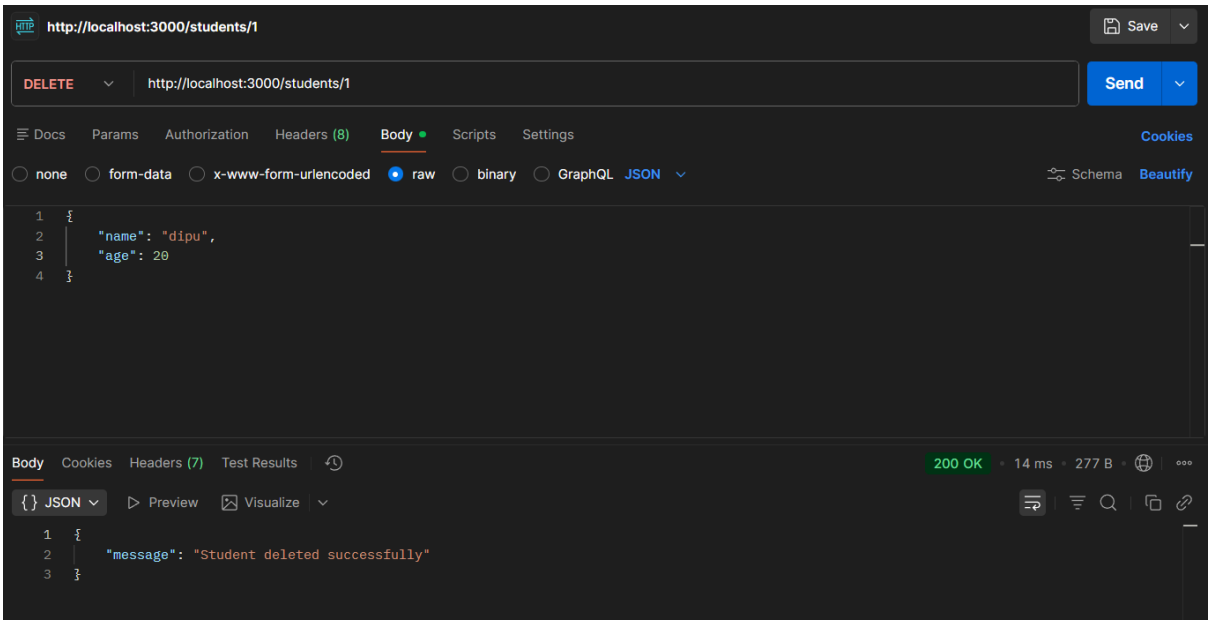
Testing the PATCH Request



Testing the PUT Request



Testing the DELETE Request



Angular.js
Practical no. :- 05
Aim: One page application on adding students data in angular

save as sakshee.html

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
<meta charset="UTF-8">
<title>Student Management</title>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/ang
ular.min.js"></script>

<style>
body{font-family:Arial;padding:20px}
input,button{margin:5px;padding:6px}
table{width:100%;border-collapse:collapse;margin-top:15px}
th,td{border:1px solid #ccc;padding:8px;text-align:center}
th{background:#007bff;color:#fff}
</style>
</head>

<body ng-controller="StudentCtrl">

<h2>Student Management</h2>

<input ng-model="student.name" placeholder="Name">
<input ng-model="student.age" type="number" placeholder="Age">
<input ng-model="student.course" placeholder="Course">

<button ng-click="save()">{{edit?'Update':'Add'}}</button>
<button ng-click="clear()">Clear</button>

<table>
<tr>
<th>#</th><th>Name</th><th>Age</th><th>Course</th><th>Action</
th>
</tr>
<tr ng-repeat="s in students track by $index">
<td>{{$index+1}}</td>
<td>{{s.name}}</td>
<td>{{s.age}}</td>
```

```
<td>{{s.course}}</td>
<td>
<button ng-click="editStudent($index)">Edit</button>
<button ng-click="students.splice($index,1)">Delete</button>
</td>
</tr>
</table>

<script>
angular.module('studentApp',[])
.controller('StudentCtrl',function($scope){
$scope.students=[];
$scope.student={};
$scope.edit=false;
$scope.i=null;

$scope.save=function(){
if($scope.edit)
$scope.students[$scope.i]=angular.copy($scope.student);
else
$scope.students.push(angular.copy($scope.student));
$scope.clear();
};

$scope.editStudent=function(i){
$scope.student=angular.copy($scope.students[i]);
$scope.i=i;$scope.edit=true;
};

$scope.clear=function(){
$scope.student={};$scope.edit=false;
};
});
</script>

</body>
</html>
```

File /home/student/Downloads/sakshee.html New Chrome available

Student Management

Name | Age | Course | Add | Clear

#	Name	Age	Course	Action
1	Sakshee	19	photoshop	Edit Delete

Student Management

Sakshee | 20 | powerbi | Update | Clear

#	Name	Age	Course	Action
1	Sakshee	20	powerbi	Edit Delete

MongoDB
Practical no. :- 06

AIM :- Student Management System (CRUD Operations)

You can run MongoDB using:

MongoDB Shell (mongosh)

After installing MongoDB, open your terminal (Command Prompt / PowerShell / Mac Terminal) and type:

```
mongosh
```

If the MongoDB server is running, it will connect.

Step 1: Create / Switch Database

```
use schoolDB
```

Output:-

```
test> use schoolDB
switched to db schoolDB
```

If the database doesn't exist, MongoDB creates it automatically when data is inserted.

Step 2: Create Collection & Insert Data

```
db.students.insertOne({
  name: "Rahul",
  age: 20,
  course: "BCA",
  marks: 75
})
```

Output:-

```
{
  acknowledged: true,
  insertedId: ObjectId('699584fec8069731bd89b03d')
}
```

Insert multiple students:

```
db.students.insertMany([
  { name: "Anita", age: 21, course: "BCA", marks: 82 },
  { name: "Vikram", age: 22, course: "BSc", marks: 65 },
  { name: "Priya", age: 20, course: "BCom", marks: 90 }
])
```

Output:-

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('699585b9c8069731bd89b03e'),
    '1': ObjectId('699585b9c8069731bd89b03f'),
    '2': ObjectId('699585b9c8069731bd89b040')
  }
}
```

Explanation:

- students → collection
- Each student is a document
- MongoDB stores data in JSON-like format (BSON)

Step 3: Read Data

Find all students:

```
db.students.find()
```

Output:-

```
schoolDB> db.students.find()
[
  {
    _id: ObjectId('699584fec8069731bd89b03d'),
    name: 'Rahul',
    age: 20,
    course: 'BCA',
    marks: 75
  },
  {
    _id: ObjectId('699585b9c8069731bd89b03e'),
    name: 'Anita',
    age: 21,
    course: 'BCA',
    marks: 82
  },
  {
    _id: ObjectId('699585b9c8069731bd89b03f'),
    name: 'Vikram',
    age: 22,
    course: 'BSc',
    marks: 65
  },
  {
    _id: ObjectId('699585b9c8069731bd89b040'),
    name: 'Priya',
    age: 20,
    course: 'BCom',
    marks: 90
  }
]
```

Find specific student:

db.students.find({ name: "Rahul" })

Output:-

```
[
  {
    _id: ObjectId('699584fec8069731bd89b03d'),
    name: 'Rahul',
    age: 20,
    course: 'BCA',
    marks: 75
  }
]
```

Find students with marks > 70:

db.students.find({ marks: { $gt: 70 } })

Output:-

```
schoolDB> db.students.find({marks: {$gt: 70}})
[
  {
    _id: ObjectId('699584fec8069731bd89b03d'),
    name: 'Rahul',
    age: 20,
    course: 'BCA',
    marks: 75
  },
  {
    _id: ObjectId('699585b9c8069731bd89b03e'),
    name: 'Anita',
    age: 21,
    course: 'BCA',
    marks: 82
  },
  {
    _id: ObjectId('699585b9c8069731bd89b040'),
    name: 'Priya',
    age: 20,
    course: 'BCom',
    marks: 90
  }
]
```

$gt means greater than.

Step 4: Update Data

Update Rahul's marks:

```
db.students.updateOne(
  { name: "Rahul" },
  { $set: { marks: 80 } }
)
```

Output:-

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

$set modifies only specific fields.

Step 5: Delete Data

Delete one student:

db.students.deleteOne({ name: "Vikram" })

Output:-

```
schoolDB> db.students.deleteOne({name: "Vikram"})
{ acknowledged: true, deletedCount: 1 }
```

Delete all BCA students:

db.students.deleteMany({ course: "BCA" })

Output:-

```
schoolDB> db.students.deleteMany({course: "BCA"})
{ acknowledged: true, deletedCount: 2 }
```

Delete all:

db.students.deleteMany({})

Output:-

```
schoolDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
```

Practical no.:- 07
Aim :- Perform Query Operations with Operators

🎯 Objective:
To use comparison and logical operators in MongoDB.

🔹 Insert Sample Data

```
db.employees.insertMany([
  { name: "Amit", salary: 30000, department: "IT", age: 25 },
  { name: "Neha", salary: 40000, department: "HR", age: 30 },
  { name: "Raj", salary: 50000, department: "IT", age: 35 },
  { name: "Simran", salary: 45000, department: "Finance", age: 28 }
])
```

Output:-

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('699d6f6f460e3aa77e89b03d'),
    '1': ObjectId('699d6f6f460e3aa77e89b03e'),
    '2': ObjectId('699d6f6f460e3aa77e89b03f'),
    '3': ObjectId('699d6f6f460e3aa77e89b040')
  }
}
```

Comparison Operators

➢ Salary greater than 40000

```
db.employees.find({ salary: { $gt: 40000 } })
```

```
[
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03f'),
    name: 'Raj',
    salary: 50000,
    department: 'IT',
    age: 35
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b040'),
    name: 'Simran',
    salary: 45000,
    department: 'Finance',
    age: 28
  }
]
```

➢ Age less than or equal to 30

```
db.employees.find({ age: { $lte: 30 } })
```

```
[
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03d'),
    name: 'Amit',
    salary: 30000,
    department: 'IT',
    age: 25
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03e'),
    name: 'Neha',
    salary: 40000,
    department: 'HR',
    age: 30
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b040'),
    name: 'Simran',
    salary: 45000,
    department: 'Finance',
    age: 28
  }
]
```

🔹 Logical Operators

➢ AND condition

```
db.employees.find({
  $and: [
    { department: "IT" },
    { salary: { $gt: 35000 } }
  ]
})
```

```
[
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03f'),
    name: 'Raj',
    salary: 50000,
    department: 'IT',
    age: 35
  }
]
```

➢ OR condition

```
db.employees.find({
  $or: [
    { department: "HR" },
    { department: "Finance" }
  ]
})
```

```
[
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03e'),
    name: 'Neha',
    salary: 40000,
    department: 'HR',
    age: 30
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b040'),
    name: 'Simran',
    salary: 45000,
    department: 'Finance',
    age: 28
  }
]
```

Sorting

```
db.employees.find().sort({ salary: -1 })
```

```
[
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03f'),
    name: 'Raj',
    salary: 50000,
    department: 'IT',
    age: 35
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b040'),
    name: 'Simran',
    salary: 45000,
    department: 'Finance',
    age: 28
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03e'),
    name: 'Neha',
    salary: 40000,
    department: 'HR',
    age: 30
  },
  {
    _id: ObjectId('699d6f6f460e3aa77e89b03d'),
    name: 'Amit',
    salary: 30000,
    department: 'IT',
    age: 25
  }
]
```

👉 `-1` = Descending order

Result:

You learned filtering, logical operators, and sorting.

Practical 8

To perform aggregation like sum, average, and grouping.

Insert Sales Data

```
db.sales.insertMany([
  { product: "Laptop", category: "Electronics", amount: 50000 },
  { product: "Mobile", category: "Electronics", amount: 20000 },
  { product: "Shoes", category: "Fashion", amount: 3000 },
  { product: "T-Shirt", category: "Fashion", amount: 1500 },
  { product: "TV", category: "Electronics", amount: 40000 }
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('699d70e8460e3aa77e89b041'),
    '1': ObjectId('699d70e8460e3aa77e89b042'),
    '2': ObjectId('699d70e8460e3aa77e89b043'),
    '3': ObjectId('699d70e8460e3aa77e89b044'),
    '4': ObjectId('699d70e8460e3aa77e89b045')
  }
}
```

Total Sales by Category

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category",
      totalSales: { $sum: "$amount" }
    }
  }
])
```

```
[
  { _id: 'Fashion', totalSales: 4500 },
  { _id: 'Electronics', totalSales: 110000 }
]
```

`$group` groups documents by category.
`$sum` calculates total.

Average Sales by Category

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category",
      avgSales: { $avg: "$amount" }
    }
  }
])
```

```
...
[
  { _id: 'Fashion', avgSales: 2250 },
  { _id: 'Electronics', avgSales: 36666.666666666664 }
]
```

Count Number of Products per Category

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category",
      count: { $sum: 1 }
    }
  }
])
```

```
[ { _id: 'Fashion', count: 2 }, { _id: 'Electronics',
 count: 3 } ]
```

Result:

You successfully performed aggregation using $group, $sum, and $avg.